WebSocket API Documentation

Overview

This document details the various commands and responses available through the WebSocket API for the chat application. The API enables real-time communication between the client and the server, supporting features like chat messaging, conversation management, image handling, humanized text generation, and more.

Connection Establishment

To establish a WebSocket connection, clients must provide a valid refresh token as a query parameter:

```
const socket = new WebSocket('wss://api.example.com/ws?token=YOUR_REFRESH_TOKEN');
```

Basic Message Structure

All messages sent to and received from the server follow a JSON format with a `type` field indicating the message purpose.

Commands and Responses

1. Chat Messaging

Send Chat Message

Send a text message to initiate or continue a conversation.

Request:

```
{
  "type": "chat",
  "content": "Hello, can you help me with a question?",
  "conversationId": "optional-existing-conversation-id",
  "stream": true
}
```

Parameters:

- `content` (required): The text content of the message
- `conversationId` (optional): ID of an existing conversation, if continuing one
- `stream` (optional, default: true): Whether to receive streaming response

Responses:

If it's a new conversation (no conversationId provided):

```
{
  "type": "conversation_created",
  "conversation_id": "new-conversation-uuid",
  "title": "Auto-generated conversation title",
  "message_id": "user-message-uuid"
}
```

For streaming response (multiple chunks):

```
{
  "type": "chat_stream",
  "conversation_id": "conversation-uuid",
  "message_id": "assistant-message-uuid",
  "chunk": "Partial text response",
  "done": false
}
```

Final streaming chunk:

```
{
  "type": "chat_stream",
  "conversation_id": "conversation-uuid",
  "message_id": "assistant-message-uuid",
  "chunk": "",
  "done": true
}
```

For non-streaming response (single message):

```
{
  "type": "chat_complete",
  "conversation_id": "conversation-uuid",
  "message_id": "assistant-message-uuid",
  "user_message_id": "user-message-uuid",
  "title": "Conversation title",
  "content": "Complete text response",
  "done": true
}
```

2. Conversation Management

Get Conversation History

Retrieve message history for a specific conversation.

Request:

```
{
  "type": "get_history",
  "conversationId": "conversation-uuid",
  "limit": 20,
  "offset": 0
}
```

Parameters:

- `conversationId` (required): The ID of the conversation to retrieve
- `limit` (optional, default: 20): Maximum number of messages to return
- `offset` (optional, default: 0): Number of messages to skip

Response:

```
{
  "type": "history",
  "conversationId": "conversation-uuid",
  "messages": [
    {
      "id": "message-uuid",
      "role": "user",
      "content": "User message content",
      "created_at": "2023-09-15T12:34:56Z",
      "tokens_used": 0,
      "has_attachments": false,
      "images": null,
      "humanized_count": 0
    },
    {
      "id": "message-uuid",
      "role": "assistant",
      "content": "Assistant response content",
      "created_at": "2023-09-15T12:35:10Z",
      "tokens_used": 150,
      "has_attachments": false,
      "images": null,
      "humanized_count": 0
    }
  ]
}
```


Get All Conversations

Retrieve a list of all conversations for the current user.

Request:


```
{
  "type": "get_conversations"
}
```


Response:


```
{
  "type": "conversations",
  "conversations": [
    {
      "id": "conversation-uuid",
      "title": "Conversation Title",
      "created_at": "2023-09-15T12:34:56Z",
      "updated_at": "2023-09-15T13:45:20Z",
      "last_message": "Last message in conversation",
      "message_count": 10
    }
  ]
}
```

Rename Conversation

Change the title of an existing conversation.

Request:

```
{
  "type": "rename_conversation",
  "conversationId": "conversation-uuid",
  "title": "New Conversation Title"
}
```

Parameters:

- `conversationId` (required): The ID of the conversation to rename
- `title` (required): The new title

Response:

```
{
  "type": "conversation_renamed",
  "conversation": {
    "id": "conversation-uuid",
    "title": "New Conversation Title",
    "created_at": "2023-09-15T12:34:56Z",
    "updated_at": "2023-09-15T14:30:00Z"
  }
}
```

Delete Conversation

Delete an existing conversation.

Request:

```
{
  "type": "delete_conversation",
  "conversationId": "conversation-uuid"
}
```

Parameters:

- `conversationId` (required): The ID of the conversation to delete

Response:

```
{
  "type": "conversation_deleted",
  "conversationId": "conversation-uuid",
  "success": true
}
```

3. Image Handling

Upload Image

Upload an image to the server. Unlike other commands, this sends binary data directly through the WebSocket.

Client Implementation:

```
// Example code for uploading an image via WebSocket
function uploadImage(websocket, file) {
  return new Promise((resolve, reject) => {
    // Create a handler for the server response
    const messageHandler = (event) => {
      try {
        const response = JSON.parse(event.data);

        if (response.type === 'error') {
          websocket.removeEventListener('message', messageHandler);
          reject(new Error(response.error));
          return;
        }

        if (response.type === 'image_uploaded') {
          websocket.removeEventListener('message', messageHandler);
          resolve(response);
          return;
        }
      } catch (error) {
        console.error('Error parsing response:', error);
      }
    };

    // Add the response handler
    websocket.addEventListener('message', messageHandler);

    // Read the file as ArrayBuffer
    const reader = new FileReader();

    reader.onload = (e) => {
      // Send the binary data directly
      websocket.send(e.target.result);
    };

    reader.onerror = (error) => {
      websocket.removeEventListener('message', messageHandler);
      reject(error);
    };

    // Read the file as binary data
    reader.readAsArrayBuffer(file);
  });
}

// Usage example:
const fileInput = document.getElementById('image-upload');
fileInput.addEventListener('change', async (event) => {
```

```
  if (event.target.files.length > 0) {
    const file = event.target.files[0];

    try {
      // Check file size (optional validation)
      const maxSize = 10 * 1024 * 1024; // 10MB
      if (file.size > maxSize) {
        alert(`File size exceeds maximum (${maxSize / 1024 / 1024}MB)`);
        return;
      }

      // Check file type (optional validation)
      if (!file.type.startsWith('image/')) {
        alert('Only image files are supported');
        return;
      }

      // Upload the image
      const result = await uploadImage(websocket, file);
      console.log('Image uploaded successfully:', result);

      // You can now use result.image_id in other operations
    } catch (error) {
      console.error('Image upload failed:', error);
    }
  }
});
```

Response:

```
{
  "type": "image_uploaded",
  "image_id": "image-uuid",
  "url": "https://presigned-s3-url-to-image.jpg",
  "filename": "original-filename.jpg",
  "size": 1024567,
  "mime_type": "image/jpeg"
}
```

Get Image Data

Retrieve a specific image.

Request:

```
{
  "type": "get_image",
  "image_id": "image-uuid"
}
```

Parameters:

- `image_id` (required): The ID of the image to retrieve

Response:

```
{
  "type": "image_data",
  "id": "image-uuid",
  "url": "https://presigned-s3-url-to-image.jpg",
  "filename": "original-filename.jpg",
  "mime_type": "image/jpeg"
}
```

Get User Images

Retrieve all images uploaded by the current user.

Request:

```
{
  "type": "get_user_images",
  "limit": 20,
  "offset": 0
}
```

Parameters:

- `limit` (optional, default: 20): Maximum number of images to return
- `offset` (optional, default: 0): Number of images to skip

Response:

```
{
  "type": "user_images",
  "images": [
    {
      "id": "image-uuid",
      "filename": "filename.jpg",
      "type": "image/jpeg",
      "size": 1024567,
      "source": "upload",
      "source_id": null,
      "created_at": "2023-09-20T15:20:30Z",
      "thumbnail": "base64-encoded-thumbnail",
      "url": "https://presigned-s3-url-to-image.jpg",
      "derived_count": 0
    }
  ]
}
```

Delete Image

Delete a user image.

Request:

```
{
  "type": "delete_image",
```

```
  "image_id": "image-uuid"
}
```

Parameters:

- `image_id` (required): The ID of the image to delete

Response:

```
{
  "type": "image_deleted",
  "image_id": "image-uuid",
  "success": true
}
```

Analyze Image

Request AI analysis of an uploaded image.

Request:

```
{
  "type": "image_analysis",
  "image_id": "image-uuid",
  "prompt": "Describe what's in this image",
  "conversationId": "optional-conversation-id"
}
```

Parameters:

- `image_id` (required): The ID of the image to analyze
- `prompt` (required): Text prompt describing what to analyze about the image
- `conversationId` (optional): ID of a conversation to add the analysis to

Responses:

Processing acknowledgment:

```
{
  "type": "image_analysis_started",
  "message_id": "message-uuid",
  "conversation_id": "conversation-uuid"
}
```

Analysis completion:

```
{
  "type": "image_analysis_complete",
  "message_id": "message-uuid",
  "conversation_id": "conversation-uuid",
  "content": "AI analysis of the image content",
  "image": {
    "id": "image-uuid",
    "thumbnail": "base64-encoded-thumbnail"
  }
}
```

Generate Image from Text

Generate an image based on a text prompt.

Request:

```
{
  "type": "chat_with_image",
  "content": "A sunset over the mountains with a lake in the foreground",
  "conversationId": "optional-conversation-id",
  "stream": true
}
```

Parameters:

- `content` (required): Text prompt describing the image to generate
- `conversationId` (optional): ID of a conversation to add the generated image to
- `stream` (optional, default: true): Whether to receive streaming response for status updates

Responses:

For streaming updates:

```
{
  "type": "chat_stream",
  "conversation_id": "conversation-uuid",
  "message_id": "assistant-message-uuid",
  "chunk": "I'll generate an image based on your prompt: \"A sunset over the mountains with a lake in the foreground\". Please wait a moment...",
  "done": false
}
```

Final response with image:

```
{
  "type": "chat_stream",
  "conversation_id": "conversation-uuid",
  "message_id": "assistant-message-uuid",
  "chunk": "Here's the image I generated based on your prompt: \"A sunset over the mountains with a lake in the foreground\"",
  "done": true,
  "image": {
    "id": "image-uuid",
    "thumbnail": "base64-encoded-thumbnail",
    "filename": "sunset-mountains-lake.png",
    "url": "https://presigned-s3-url-to-image.png"
  }
}
```

4. Text Humanization

Humanize Message or Text

Transform AI-generated text to appear more human-written.

Request for humanizing a specific message:

```
{
  "type": "humanize",
  "message_id": "message-uuid",
  "request_id": "client-generated-request-id"
}
```

Request for humanizing arbitrary text:

```
{
  "type": "humanize",
  "text": "Text to be humanized",
  "request_id": "client-generated-request-id"
}
```

Parameters:

- `message_id` (required for message humanization): ID of the message to humanize
- `text` (required for direct text humanization): Text to be humanized
- `request_id` (optional): Client-generated ID to track the request
- `re_humanize` (optional, default: false): Whether to re-humanize already humanized text

Responses:

Processing acknowledgment:

```
{
  "type": "humanize_status",
  "status": "processing",
  "request_id": "client-generated-request-id"
}
```

For message humanization completion:

```
{
  "type": "humanize_complete",
  "original_text": "Original message text",
  "humanized_text": "Humanized version of the text",
  "request_id": "client-generated-request-id",
  "message_id": "message-uuid"
}
```

For direct text humanization completion:

```
{
  "type": "text_humanize_complete",
  "original_text": "Original input text",
  "humanized_text": "Humanized version of the text",
  "request_id": "client-generated-request-id"
}
```

Get Humanized Messages

Retrieve humanized versions of messages.

Request:

```
{
  "type": "get_humanized_messages",
  "message_id": "optional-specific-message-id",
  "limit": 20,
  "offset": 0
}
```

Parameters:

- `message_id` (optional): ID of a specific message to get humanized versions for
- `limit` (optional, default: 20): Maximum number of humanized messages to return
- `offset` (optional, default: 0): Number of humanized messages to skip

Response:

```
{
  "type": "get_humanized_messages",
  "messageId": "message-uuid-or-null",
  "messages": [
    {
      "id": "humanized-message-uuid",
      "humanized_text": "Humanized text content",
      "created_at": "2023-09-18T10:20:30Z",
      "re_humanized": false
    }
  ]
}
```

Error Handling

All commands can return an error response:

```
{
  "type": "error",
  "error": "Error message description",
  "request_id": "client-generated-request-id",
  "conversationId": "conversation-uuid",
  "message_id": "message-uuid"
}
```

WebSocket Lifecycle

Connection Close Codes

The server may close the connection with the following codes:

- `4000`: General connection error
- `4001`: Unauthorized (invalid or expired token)
- `4003`: Rate limit exceeded

Best Practices

1. **Error Handling**: Always implement error handling for all WebSocket operations.
2. **Reconnection Logic**: Implement automatic reconnection with backoff strategy.
3. **Message Tracking**: Use request IDs to track asynchronous operations.
4. **Resource Management**: Close the WebSocket connection when not in use.
5. **Image Uploads**: Validate image types and sizes before uploading.
6. **Timeout Handling**: Implement timeouts for long-running operations.